
Laboratoire de High Performance Coding

semestre printemps 2026

Laboratoire 5 : Profiling

Temps à disposition : 4 périodes (2 séances de laboratoire).

1 Objectifs de ce laboratoire

L'objectif de ce laboratoire est de se familiariser avec des outils de profiling afin d'analyser les performances d'un programme. Pour cela, vous allez utiliser les outils proposés pour profiler des programmes.

2 Perf

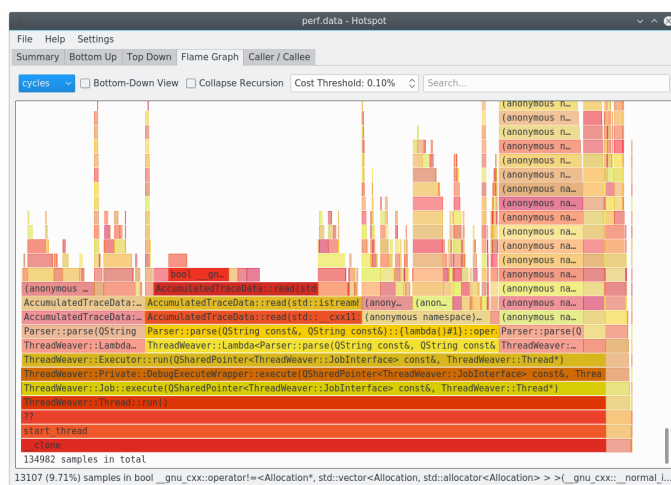
Le kernel Linux implémente des moyens de profiling, accessibles depuis l'outil `perf`. Beaucoup de ressources en ligne décrivent l'architecture et l'utilisation de `perf`. Pour vous familiariser avec cet outil, vous pouvez suivre le tutoriel en trois étapes qui décrit son utilisation, ainsi que consulter cette source d'informations.

Vous pouvez utiliser les commandes `perf stat` et `perf top` pour commencer votre analyse, puis utiliser `perf record` et `perf report` pour approfondir l'analyse.

Pour plus d'informations techniques sur `perf` (facultatif), vous pouvez consulter sa documentation officielle.

3 Hotspot

`perf` ne permet pas de visualiser facilement les "hotspots" d'un programme qui a été profilé. Ces hotspots peuvent être visualisés sous la forme d'un "flame graph", comme celui proposé par le programme "Hotspot" ci-dessous :



Pour vous familiariser avec Hotspot, vous pouvez lire la page README de son dépôt Git, puis essayer le programme. Vous pouvez effectuer des mesures de cycles mais également de cache misses.

4 Valgrind

Valgrind est un outil sous Linux initialement conçu pour le débogage de l'allocation dynamique de mémoire sur le heap (memcheck). Valgrind a depuis évolué en une suite d'outils (helgrind, cachegrind, callgrind, ...) permettant également de faire du profiling. Pour utiliser les outils cachegrind et callgrind, vous pouvez suivre ce tutoriel : [Guide to Callgrind](#).

Si vous souhaitez utiliser l'outil memcheck, vous pouvez consulter la documentation officielle à cette adresse : [Manual for Memcheck](#) (facultatif).

5 Travail à effectuer

5.1 Étude de cas 1 : accès mémoire séquentiels et aléatoires avec `perf stat`

Dans cette première partie, nous allons utiliser `perf stat` afin de comparer plusieurs modes d'exécution d'un même programme C (le `case1`). L'objectif est de comprendre comment des compteurs matériels permettent de caractériser le comportement d'un programme.

Le programme fourni propose trois modes d'exécution :

- `seq` : parcours séquentiel d'un tableau ;
- `rand` : parcours du même tableau via une permutation aléatoire ;

Les trois modes calculent une somme sur les mêmes données, mais ne parcourent pas la mémoire de la même manière.

Avant de commencer Avant toute mesure, formulez une hypothèse :

- quel mode devrait être le plus rapide ;
- quelle pourrait être la cause principale de la différence observée.

Mesures de base Commencez par exécuter les commandes suivantes :

```
perf stat ./case1 seq
perf stat ./case1 rand
```

Mesures détaillées Vous pouvez également demander à `perf` de mesurer des événements plus précis, en fonction de ce que vous souhaitez observer. Cela doit vous permettre d'identifier plus clairement la différence entre les deux modes d'exécution.

Conclusion Avec l'exécution des commande `perf stat` vous devez être capable de comprendre si un des deux modes est plus "bounded" que l'autre et surtout les raison pour lesquels il le serait.

5.2 Étude de cas 2 : identification des hotspots avec `perf` et Hotspot

Dans cette seconde partie, nous allons nous intéresser non plus à la caractérisation globale des performances, mais à la localisation des parties du code responsables du temps d'exécution.

Pour cela, nous utiliserons les outils `perf record` et `perf report`, ainsi que l'interface graphique Hotspot.

Contrairement à `perf stat`, qui fournit des compteurs globaux, ces outils permettent d'identifier les fonctions qui consomment le plus de temps CPU : les *hotspots*.

Programme étudié Le programme fourni (`case2`) applique un filtre simple (flou) sur une image. Ce traitement est répété plusieurs fois afin de rendre son coût mesurable.

Avant de commencer Avant toute mesure, examinez brièvement le code source et réfléchissez aux questions suivantes :

- quelle(s) fonction(s) vous semblent les plus coûteuses ;
- où se situe, selon vous, la majorité du temps d'exécution.

Analyse avec perf La commande `perf top` permet d'observer en temps réel les fonctions consommant le plus de CPU. Elle effectue un échantillonnage et affiche les fonctions les plus actives à un instant donné.

Cependant, cet outil est difficile à utiliser proprement pour analyser un programme court, et les résultats peuvent être bruités. Nous utiliserons donc plutôt une approche basée sur l'enregistrement complet d'une exécution.

Enregistrez une exécution complète du programme :

```
perf record -g ./case2
```

Puis analysez les résultats :

```
perf report
```

Vous devriez pouvoir observer votre code annoté, et identifier relativement facilement où le programme passe la majorité de son temps.

Analyse avec Hotspot Chargez le fichier généré par `perf record` dans Hotspot (l'utilisation de `sudo` peut être nécessaire) :

```
hotspot perf.data
```

Vous pourrez notamment observer un *flame graph*, qui représente visuellement la répartition du temps d'exécution dans les différentes fonctions.

Lecture du flame graph Dans un flame graph :

- la largeur d'un bloc correspond au temps passé dans la fonction ;
- la hauteur représente la profondeur de la pile d'appels ;
- les blocs les plus larges correspondent aux hotspots.

Conclusion Grâce à `perf record` et `perf report`, ainsi qu'à l'outil Hotspot, vous devez être capables d'identifier précisément les parties du code où le temps d'exécution est concentré.

Cette étape est essentielle avant toute optimisation : il est inutile d'optimiser du code qui ne représente qu'une faible fraction du temps total. L'objectif ici est donc de comprendre comment localiser efficacement un hotspot, et de pouvoir proposer une piste d'amélioration basée sur cette observation.

5.3 Étude de cas 3 : impact de la localité mémoire avec `cachegrind`

Dans cette troisième partie, nous allons approfondir l'analyse des performances en nous intéressant à l'utilisation du cache.

Alors que `perf stat` permet de caractériser globalement un programme, et que `perf record` permet d'identifier les zones coûteuses, l'outil `cachegrind` (permettant de simuler le comportement du cache) permet de comprendre plus finement l'impact des accès mémoire.

Programme étudié Le programme fourni (case3) implémente deux versions d'une multiplication de matrices :

- une version `ijk` classique ;
- une version `ikj` où l'ordre des boucles est modifié.

Ces deux versions ont la même complexité algorithmique ($O(n^3)$), mais leur comportement vis-à-vis du cache peut être très différent.

Avant de commencer Avant toute mesure, examinez le code source et réfléchissez aux questions suivantes :

- les deux versions effectuent-elles le même nombre d'opérations ?
- laquelle vous semble la plus efficace du point de vue des accès mémoire ?

Mesure avec cachegrind Exécutez le programme avec `cachegrind` :

```
valgrind --tool=cachegrind ./case3
```

Un fichier de sortie `cachegrind.out.<pid>` est alors généré.

Analysez ensuite ce fichier avec :

```
cg_annotate cachegrind.out.<pid>
```

Avec perf Pouvez vous avec `perf` voir le même problème, regardez la liste des compteurs disponible.

Analyse Contrairement à `perf`, `cachegrind` ne mesure pas directement le temps d'exécution, mais fournit une estimation précise du nombre d'instructions exécutées et du comportement du cache.

Dans la sortie annotée, vous pouvez observer le nombre d'instructions exécutées (`Ir`) pour chaque ligne du programme.

Essayez d'identifier :

- les boucles les plus coûteuses ;
- les différences de comportement entre les versions `ijk` et `ikj`.

Interprétation Même si les deux versions effectuent un nombre similaire d'opérations, leur ordre d'accès aux données est différent.

Réfléchissez à la manière dont les matrices sont stockées en mémoire (ordre ligne par ligne), et à la façon dont chaque version parcourt ces données.

Conclusion Vous devez être capables de comprendre que deux algorithmes de même complexité peuvent avoir des performances très différentes en pratique, en raison de leur localité mémoire.

L'objectif ici est de mettre en évidence l'importance du cache dans les performances des programmes, et de comprendre comment l'ordre des boucles peut influencer fortement ces performances.

5.4 Étude de cas 4 : analyse du graphe d'appels avec `callgrind`

Dans cette dernière partie, nous allons analyser la structure des appels d'un programme, afin de comprendre comment le temps d'exécution se répartit entre les différentes fonctions.

Pour cela, nous utiliserons l'outil `callgrind`, qui permet de générer un graphe d'appels détaillé, indiquant combien de fois chaque fonction est appelée et combien d'instructions sont exécutées.

Contrairement aux outils précédents :

- `perf stat` caractérise globalement le programme ;
- `perf record` permet d'identifier les zones coûteuses ;
- `cachegrind` permet d'analyser l'impact du cache ;

`callgrind` permet quant à lui de comprendre comment ces coûts se propagent dans le graphe d'appels.

Programme étudié Le programme fourni (case4) calcule la suite de Fibonacci de manière récursive. Ce type d'implémentation est volontairement inefficace, afin de mettre en évidence la structure des appels.

Avant de commencer Avant toute mesure, examinez le code source et réfléchissez aux points suivants :

- combien d'appels à la fonction récursive sont effectués ;
- comment le nombre d'appels évolue avec la valeur de `n`.

Mesure avec `callgrind` Exécutez le programme avec `callgrind` :

```
valgrind --tool=callgrind ./case4
```

Un fichier de sortie `callgrind.out.<pid>` est généré.

Analysez ensuite ce fichier avec :

```
callgrind_annotate callgrind.out.<pid>
```

Comparaison avec `perf` Vous pouvez également analyser ce programme avec `perf` :

```
perf record -g ./case4
perf report
```

Comparez les résultats obtenus avec ceux de `callgrind`.

Essayez d'identifier :

- si les mêmes fonctions apparaissent comme les plus coûteuses ;
- si la répartition du coût est similaire.

Analyse La sortie de `callgrind` indique le nombre d'instructions exécutées (`Ir`) pour chaque fonction, ainsi que la répartition du coût entre les appels.

Observez en particulier :

- la fonction la plus coûteuse ;
- la proportion du temps total qu'elle représente ;
- le nombre très élevé d'appels récursifs.

Interprétation Dans ce programme, la majorité du temps est passée dans la fonction récursive, non pas parce qu'un appel individuel est particulièrement coûteux, mais parce que cette fonction est appelée un très grand nombre de fois.

Cela met en évidence une distinction importante :

- une fonction peut être peu coûteuse individuellement ;
- mais devenir dominante si elle est appelée très fréquemment.

Conclusion Vous devez être capables de comprendre que les performances d'un programme peuvent être fortement influencées par sa structure d'appels.

L'objectif ici est d'identifier les cas où un algorithme entraîne un nombre d'appels excessif, et de comprendre que l'optimisation peut parfois nécessiter une modification de l'algorithme lui-même (par exemple en évitant des appels redondants).

5.5 Conclusion générale

À travers ces quatre études de cas, vous avez utilisé différents outils de profiling pour analyser un programme sous plusieurs angles : comportement global, localisation des hotspots, impact de la localité mémoire, et structure du graphe d'appels.

L'objectif de ce laboratoire est de vous montrer qu'une bonne optimisation ne consiste pas à modifier du code au hasard, mais à observer, mesurer, interpréter, puis agir sur les véritables causes du coût d'exécution.

5.6 Prise en main

Afin de vous familiariser avec les outils présentés dans ce laboratoire, nous vous fournissons un programme que vous pouvez analyser.

Le code disponible dans le dossier `code` a pour objectif de calculer la température moyenne par ville, ainsi que les températures minimale et maximale pour chaque ville. Chaque ligne du fichier de données est au format suivant : `Ville;température`.

Après compilation du projet, deux exécutables sont disponibles :

- `build/create_sample` : génère un fichier de données contenant des mesures de température. Il prend en argument un entier N correspondant au nombre de lignes du fichier généré.
- `build/analyse` : analyse le fichier de données et calcule les températures minimale, maximale et moyenne pour chaque ville. Il prend en argument le chemin vers le fichier généré.

Votre objectif est d'utiliser les outils étudiés dans ce laboratoire afin d'analyser le comportement de ce programme.

En particulier, vous devrez :

- caractériser globalement le programme (par exemple avec `perf stat`) ;
- identifier les fonctions ou parties du code les plus coûteuses ;

- analyser l'impact des accès mémoire si nécessaire ;
- interpréter les résultats obtenus.

Il ne vous est pas demandé, dans cette partie, de modifier ou d'optimiser le code, mais d'être capable de comprendre et d'expliquer les comportements observés.

5.7 Profiling

Dans cette seconde partie, vous devez appliquer la démarche de profiling sur un projet que vous connaissez déjà.

Nous vous demandons de reprendre votre programme du laboratoire 01 et d'en analyser les performances à l'aide des outils étudiés (`perf`, Hotspot, Valgrind).

L'objectif est de suivre une démarche structurée :

- définir un scénario d'exécution représentatif ;
- mesurer les performances globales ;
- identifier les zones coûteuses ;
- analyser les causes possibles (calcul, mémoire, structure des appels) ;
- proposer une ou plusieurs pistes d'amélioration.

6 Travail à rendre

Vous devrez remettre un rapport présentant votre analyse.

Ce rapport devra contenir :

- une analyse du programme fourni (partie 5.6), en vous appuyant sur les outils de profiling utilisés dans le laboratoire ;
- une analyse de votre programme du laboratoire 01, incluant les mesures effectuées et les problèmes identifiés ;
- au moins une proposition d'amélioration, accompagnée d'une justification ;
- une comparaison avant/après lorsque des modifications ont été apportées, afin de mesurer objectivement leur impact.

Les conclusions doivent être basées sur des observations mesurées, et non sur des intuitions seules.